

Nand2Tetris Assignment

- Aarav Chauhan: 2501420027

Project 1

1) *NOT*

```
2)      */
3)      CHIP Not {
4)          IN in;
5)          OUT out;
6)
7)          PARTS:
8)          Nand(a=in, b=in, out=out);
9)      }
```

The screenshot displays the Nand2Tetris Hardware Simulator interface. The main window shows the HDL code for a 'Not' chip. The code defines a chip with an input 'in' and an output 'out'. It uses a NAND gate from the 'PARTS' section to implement the NOT function: `Nand(a=in, b=in, out=out);`. The simulator's top bar shows 'NAND2Tetris / Hardware Simulator / Not.hdl'. Below the code editor, there are tabs for 'HDL', 'Test', 'Compare File', 'Output File', and 'Diff Table'. The 'Test' tab is active, showing a test script with steps 5 and outputs 3. The 'Output pins' section on the right shows the input 'in' as 1 and the output 'out' as 0. A green status bar at the bottom indicates 'Simulation successful: The output file is identical to the compare file'.

NAND2Tetris / Hardware Simulator / Not.hdl

HDL --LBSt Project 1 Not

```
1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/1/Not.hdl
5 /**
6  * Not gate:
7  * if (in) out = 0, else out = 1
8  */
9 CHIP Not {
10     IN in;
11     OUT out;
12
13     PARTS:
14     Nand(a=in, b=in, out=out);
15 }
```

Chip Not Eval Reset Clock: 0

Input pins
in 1

Output pins
out 0

Test Steps: 5 Outputs: 3 Slow Fast

Test Script Compare File Output File Diff Table

```
7 compare-to Not.cmp,
8 output-list in out;
9
10 set in 0,
11 eval,
12 output;
13
14 set in 1,
15 eval,
16 output;
```

Simulation successful: The output file is identical to the compare file

2) *AND*

```
3)  CHIP And {
4)      IN a, b;
5)      OUT out;
6)
7)      PARTS:
8)      Nand(a=a , b=b , out=c);
9)      Not(in=c , out=out );
10) }
```

The screenshot displays the Nand2Tetris Hardware Simulator interface. The main window shows the HDL code for the 'And' chip in 'And.hdl'. The code defines a chip with two input pins (a, b) and one output pin (out). It uses a Nand gate and a Not gate to implement the AND function. The right sidebar shows the chip's configuration: input pins a and b are set to 1, and the output pin out is set to 0. The bottom section shows the test script, which sets a=1, b=0, evaluates the chip, and outputs the result. The test results show that the output is 0, which is correct for the AND function with inputs 1 and 0. A green message at the bottom indicates that the simulation was successful and the output file matches the compare file.

NAND2Tetris / Hardware Simulator / And.hdl

HDL --LBSt Project 1 And

Chip And Eval Reset Clock: 0

Input pins
a 1
b 1
Output pins
out -
Internal pins
c 0

Test Steps: 7 Outputs: 5 Slow Fast

Test Script Compare File Output File Diff Table

```
19
20 set a 1,
21 set b 0,
22 eval,
23 output;
24
25 set a 1,
26 set b 1,
27 eval,
28 output;
```

Simulation successful: The output file is identical to the compare file

3) OR

```
4)  CHIP Or {
5)      IN a, b;
6)      OUT out;
7)
8)      PARTS:
9)          Nand(a=a , b=a , out=c );
10)         Nand(a=b , b=b , out=d );
11)         Nand(a=c , b=d , out=out );
12) }
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The main window shows the HDL code for the Or chip, which is implemented using three Nand gates. The code is as follows:

```
1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press
4 // File name: projects/1/Or.hdl
5 /**
6  * Or gate:
7  * if (a or b) out = 1, else out = 0
8  */
9 CHIP Or {
10     IN a, b;
11     OUT out;
12
13     PARTS:
14         Nand(a=a , b=a , out=c );
15         Nand(a=b , b=b , out=d );
16         Nand(a=c , b=d , out=out );
17 }
```

The right panel shows the chip configuration for 'Chip Or'. The input pins are 'a' and 'b', both set to 1. The output pin is 'out', set to 0. The internal pins are 'c' and 'd', both set to 0. The 'Eval' button is highlighted.

The bottom panel shows the test script and simulation results. The test script is as follows:

```
19
20 set a 1,
21 set b 0,
22 eval,
23 output;
24
25 set a 1,
26 set b 1,
27 eval,
28 output;
```

The simulation results show that the output is 0 for the first test case (a=1, b=0) and 1 for the second test case (a=1, b=1). A green status bar at the bottom indicates: "Simulation successful: The output file is identical to the compare file".

4) XOR

```
1)  CHIP Xor {
2)      IN a, b;
3)      OUT out;
4)
5)      PARTS:
6)      Nand(a=a , b=a , out=nand1out );
7)
8)      Nand(a=nand1out, b=b , out=nand2out );
9)
10)
11)
12)     Nand(a=a , b=nand4out , out=nand3out );
13)
14)     Nand(a=b , b=b , out=nand4out );
15)
16)     Nand(a=nand2out , b=nand3out , out=out );
17) }
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top bar shows the project name "NAND2Tetris" and the file "Xor.hdl". The main editor displays the HDL code for the XOR chip, which uses NAND gates to implement the XOR function. The code is as follows:

```
1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/1/Xor.hdl
5 /**
6  * Exclusive-or gate:
7  * if ((a and Not(b)) or (Not(a) and b)) out = 1, else out = 0
8  */
9 CHIP Xor {
10     IN a, b;
11     OUT out;
12
13     PARTS:
14     Nand(a=a , b=a , out=nand1out );
15
16     Nand(a=nand1out, b=b , out=nand2out );
17
18     Nand(a=a , b=nand4out , out=nand3out );
19
20     Nand(a=b , b=b , out=nand4out );
21
22     Nand(a=nand2out , b=nand3out , out=out );
23 }
```

The right panel shows the "Chip Xor" configuration with the following pins:

- Input pins: a = 1, b = 1
- Output pins: out = 0
- Internal pins: nand1out = 0, nand2out = 1, nand4out = 0, nand3out = 1

The bottom panel shows the "Test" results, indicating that the simulation was successful. The test script is as follows:

```
19
20 set a 1,
21 set b 0,
22 eval,
23 output;
24
25 set a 1,
26 set b 1,
27 eval,
28 output;
```

The status bar at the bottom confirms: "Simulation successful: The output file is identical to the compare file".

5) MUX

```
6)  CHIP Mux {
7)      IN a, b, sel;
8)      OUT out;
9)
10)     PARTS:
11)     Not(in=sel , out=x );
12)     And(a=x , b=a , out=w1 );
13)     And(a=sel , b=b , out=w2 );
14)     Or(a=w1 , b=w2 , out=out );
15) }
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top bar shows the project name "NAND2Tetris" and the current file "Mux.hdl". The main editor area contains the HDL code for the Mux chip, which is a 2-to-1 multiplexer. The code defines inputs a, b, and sel, and an output out. It uses a NOT gate to invert sel, AND gates to combine the inputs with the inverted sel, and an OR gate to produce the final output out.

On the right side, the "Chip Mux" panel shows the current state of the chip. The input pins are a=1, b=1, and sel=1. The output pin is out=0. The internal pins are x=0, w1=0, and w2=1.

At the bottom, the "Test" panel shows the test script and the simulation results. The test script sets a=1, b=1, and sel=0, then evaluates the output. The simulation results show that the output is 0, which is correct for the given inputs.

Simulation successful: The output file is identical to the compare file

6) DMUX

```
7)  CHIP DMux {
8)      IN in, sel;
9)      OUT a, b;
10)
11)  PARTS:
12)      Not(in=sel , out=nsel );
13)      And(a=in , b=nsel , out=a );
14)      And(a=in , b=sel , out=b );
15) }
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top bar shows the project name "NAND2Tetris / Hardware Simulator / DMux.hdl". The main editor area contains the HDL code for the DMux chip, which is a 2-to-1 multiplexer. A tooltip points to the "Open an HDL file using this menu" button. The right panel shows the "Chip DMux" configuration with input pins "in" (1) and "sel" (1), output pins "a" (0) and "b" (1), and an internal pin "nsel" (0). The bottom panel shows the "Test" section with a "Test Script" tab containing the following code:

```
17 output;
18
19 set in 1,
20 set sel 0,
21 eval,
22 output;
23
24 set sel 1,
25 eval,
26 output;
```

The simulation status bar at the bottom indicates "Simulation successful: The output file is identical to the compare file".

7) NOT16

```
8)    CHIP Not16 {
9)        IN in[16];
10)       OUT out[16];
11)
12)       PARTS:
13)       Not(in=in[0] , out=out[0] );
14)       Not(in=in[1] , out=out[1] );
15)       Not(in=in[2] , out=out[2] );
16)       Not(in=in[3] , out=out[3] );
17)       Not(in=in[4] , out=out[4] );
18)       Not(in=in[5] , out=out[5] );
19)       Not(in=in[6] , out=out[6] );
20)       Not(in=in[7] , out=out[7] );
21)       Not(in=in[8] , out=out[8] );
22)       Not(in=in[9] , out=out[9] );
23)       Not(in=in[10] , out=out[10] );
24)       Not(in=in[11] , out=out[11] );
25)       Not(in=in[12] , out=out[12] );
26)       Not(in=in[13] , out=out[13] );
27)       Not(in=in[14] , out=out[14] );
28)       Not(in=in[15] , out=out[15] );
29) }
```

NAND2Tetris / Hardware Simulator / Not16.hdl

HDL --LBSt Project 1 Not16

Chip Not16 Eval Reset Clock: 0

Input pins
in 0 0 0 1 0 0 1 0 0 0 1 1 0 1 0 0 dec

Output pins
out 1 1 1 0 1 1 0 1 1 1 0 0 1 0 1 1 dec

Test Steps: 8 Outputs: 6 Slow Fast

Test Script Compare File Output File Diff Table

```
19 eval,
20 output;
21
22 set in %0001110011000011,
23 eval,
24 output;
25
26 set in %00001001000110100,
27 eval,
28 output;
```

Simulation successful: The output file is identical to the compare file

8) AND16

```
9)    HIP And16 {
10)      IN a[16], b[16];
11)      OUT out[16];
12)
13)      PARTS:
14)        And(a=a[0] , b=b[0] , out=out[0] );
15)        And(a=a[1] , b=b[1] , out=out[1] );
16)        And(a=a[2] , b=b[2] , out=out[2] );
17)        And(a=a[3] , b=b[3] , out=out[3] );
18)        And(a=a[4] , b=b[4] , out=out[4] );
19)        And(a=a[5] , b=b[5] , out=out[5] );
20)        And(a=a[6] , b=b[6] , out=out[6] );
21)        And(a=a[7] , b=b[7] , out=out[7] );
22)        And(a=a[8] , b=b[8] , out=out[8] );
23)        And(a=a[9] , b=b[9] , out=out[9] );
24)        And(a=a[10] , b=b[10] , out=out[10] );
25)        And(a=a[11] , b=b[11] , out=out[11] );
26)        And(a=a[12] , b=b[12] , out=out[12] );
27)        And(a=a[13] , b=b[13] , out=out[13] );
28)        And(a=a[14] , b=b[14] , out=out[14] );
29)        And(a=a[15] , b=b[15] , out=out[15] );
30)    }
```

NAND2Tetris / Hardware Simulator / And16.hdl

HDL --LBSt Project 1 And16

10 CHIP And16 {
15 And(a=a[0] , b=b[0] , out=out[0]);
16 And(a=a[1] , b=b[1] , out=out[1]);
17 And(a=a[2] , b=b[2] , out=out[2]);
18 And(a=a[3] , b=b[3] , out=out[3]);
19 And(a=a[4] , b=b[4] , out=out[4]);
20 And(a=a[5] , b=b[5] , out=out[5]);
21 And(a=a[6] , b=b[6] , out=out[6]);
22 And(a=a[7] , b=b[7] , out=out[7]);
23 And(a=a[8] , b=b[8] , out=out[8]);
24 And(a=a[9] , b=b[9] , out=out[9]);
25 And(a=a[10] , b=b[10] , out=out[10]);
26 And(a=a[11] , b=b[11] , out=out[11]);
27 And(a=a[12] , b=b[12] , out=out[12]);
28 And(a=a[13] , b=b[13] , out=out[13]);
29 And(a=a[14] , b=b[14] , out=out[14]);
30 And(a=a[15] , b=b[15] , out=out[15]);
31 }

Chip And16 Eval Reset Clock: 0

Input pins
a 0 0 0 1 0 0 1 0 0 0 1 1 0 1 0 0 dec
b 1 0 0 1 1 0 0 0 0 0 1 1 1 0 1 1 0 dec

Output pins
out 0 0 0 1 0 0 0 0 0 0 0 1 1 0 1 0 0 dec

Test Steps: 9 Outputs: 7 Slow Fast

Test Script Compare File Output File Diff Table

```
29  
30 set a %B0011110011000011,  
31 set b %B0000111111110000,  
32 eval,  
33 output;  
34  
35 set a %B0001001000110100,  
36 set b %B1001100001110110,  
37 eval,  
38 output;
```

Simulation successful: The output file is identical to the compare file

9) OR16

```
10) CHIP Or16 {
11)     IN a[16], b[16];
12)     OUT out[16];
13)
14)     PARTS:
15)         Or(a=a[0] , b=b[0] , out=out[0] );
16)         Or(a=a[1] , b=b[1] , out=out[1] );
17)         Or(a=a[2] , b=b[2] , out=out[2] );
18)         Or(a=a[3] , b=b[3] , out=out[3] );
19)         Or(a=a[4] , b=b[4] , out=out[4] );
20)         Or(a=a[5] , b=b[5] , out=out[5] );
21)         Or(a=a[6] , b=b[6] , out=out[6] );
22)         Or(a=a[7] , b=b[7] , out=out[7] );
23)         Or(a=a[8] , b=b[8] , out=out[8] );
24)         Or(a=a[9] , b=b[9] , out=out[9] );
25)         Or(a=a[10] , b=b[10] , out=out[10] );
26)         Or(a=a[11] , b=b[11] , out=out[11] );
27)         Or(a=a[12] , b=b[12] , out=out[12] );
28)         Or(a=a[13] , b=b[13] , out=out[13] );
29)         Or(a=a[14] , b=b[14] , out=out[14] );
30)         Or(a=a[15] , b=b[15] , out=out[15] );
31) }
```

NAND2Tetris / Hardware Simulator / Or16.hdl

HDL --LBSt Project 1 Or16

10 CHIP Or16 {
15 Or(a=a[0] , b=b[0] , out=out[0]);
16 Or(a=a[1] , b=b[1] , out=out[1]);
17 Or(a=a[2] , b=b[2] , out=out[2]);
18 Or(a=a[3] , b=b[3] , out=out[3]);
19 Or(a=a[4] , b=b[4] , out=out[4]);
20 Or(a=a[5] , b=b[5] , out=out[5]);
21 Or(a=a[6] , b=b[6] , out=out[6]);
22 Or(a=a[7] , b=b[7] , out=out[7]);
23 Or(a=a[8] , b=b[8] , out=out[8]);
24 Or(a=a[9] , b=b[9] , out=out[9]);
25 Or(a=a[10] , b=b[10] , out=out[10]);
26 Or(a=a[11] , b=b[11] , out=out[11]);
27 Or(a=a[12] , b=b[12] , out=out[12]);
28 Or(a=a[13] , b=b[13] , out=out[13]);
29 Or(a=a[14] , b=b[14] , out=out[14]);
30 Or(a=a[15] , b=b[15] , out=out[15]);
31 }

Chip Or16 Eval Reset Clock: 0

Input pins
a 0 0 0 1 0 0 1 0 0 0 1 1 0 1 0 0 dec
b 1 0 0 1 1 0 0 0 0 0 1 1 1 0 1 1 0 dec

Output pins
out 1 0 0 1 1 0 1 0 0 1 1 1 0 1 1 0 dec

Test Steps: 9 Outputs: 7 Slow Fast

Test Script Compare File Output File Diff Table

```
29  
30 set a %0001110011000011,  
31 set b %00000111111110000,  
32 eval,  
33 output;  
34  
35 set a %0001001000110100,  
36 set b %100110000110110,  
37 eval,  
38 output;
```

Simulation successful: The output file is identical to the compare file

10) MUX16

```
11) CHIP Mux16 {
12)     IN a[16], b[16], sel;
13)     OUT out[16];
14)
15)     PARTS:
16)         Mux(a=a[0] , b=b[0] , sel=sel , out=out[0] );
17)         Mux(a=a[1] , b=b[1] , sel=sel , out=out[1] );
18)         Mux(a=a[2] , b=b[2] , sel=sel , out=out[2] );
19)         Mux(a=a[3] , b=b[3] , sel=sel , out=out[3] );
20)         Mux(a=a[4] , b=b[4] , sel=sel , out=out[4] );
21)         Mux(a=a[5] , b=b[5] , sel=sel , out=out[5] );
22)         Mux(a=a[6] , b=b[6] , sel=sel , out=out[6] );
23)         Mux(a=a[7] , b=b[7] , sel=sel , out=out[7] );
24)         Mux(a=a[8] , b=b[8] , sel=sel , out=out[8] );
25)         Mux(a=a[9] , b=b[9] , sel=sel , out=out[9] );
26)         Mux(a=a[10] , b=b[10] , sel=sel , out=out[10] );
27)         Mux(a=a[11] , b=b[11] , sel=sel , out=out[11] );
28)         Mux(a=a[12] , b=b[12] , sel=sel , out=out[12] );
29)         Mux(a=a[13] , b=b[13] , sel=sel , out=out[13] );
30)         Mux(a=a[14] , b=b[14] , sel=sel , out=out[14] );
31)         Mux(a=a[15] , b=b[15] , sel=sel , out=out[15] );
32) }
```

NAND2Tetris / Hardware Simulator / Mux16.hdl

HDL --LBSt Project 1 Mux16

Chip Mux16 Eval Reset Clock: 0

Input pins

a 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 dec

b 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec

sel 1

Output pins

out 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec

Test Steps: 11 Outputs: 9 Slow Fast

Test Script Run Compare File Output File Diff Table

```
39
40 set a %B1010101010101010,
41 set b %B0101010101010101,
42 set sel 0,
43 eval,
44 output;
45
46 set sel 1,
47 eval,
48 output;
```

Simulation successful: The output file is identical to the compare file

11) OR8WAY

```
12) CHIP Or8Way {
13)     IN in[8];
14)     OUT out;
15)
16)     PARTS:
17)         Or(a=in[0] , b=in[1] , out=or01 );
18)         Or(a=in[2] , b=in[3] , out=or23 );
19)         Or(a=in[4] , b=in[5] , out=or45 );
20)         Or(a=in[6] , b=in[7] , out=or67 );
21)         Or(a=or01 , b=or23 , out=part1 );
22)         Or(a=or45 , b=or67 , out=part2 );
23)         Or(a=part1 , b=part2 , out=out );
24) }
```

NAND2Tetris / Hardware Simulator / Or8Way.hdl

HDL --LBSt Project1 Or8Way

Chip Or8Way Eval Reset Clock: 0

Input pins
in 0 0 1 0 0 1 1 0 dec

Output pins
out 1

Internal pins
or01 1
or23 1
or45 1
or67 0
part1 1
part2 1

Test Steps: 8 Outputs: 6 Slow Fast

Test Script Compare File Output File Diff Table

```
19 eval,
20 output;
21
22 set in %800000001,
23 eval,
24 output;
25
26 set in %800100110,
27 eval,
28 output;
```

Simulation successful: The output file is identical to the compare file

12) MUX4WAY16

```
11 CHIP Mux4Way16 {
12     IN a[16], b[16], c[16], d[16], sel[2];
13     OUT out[16];
14
15     PARTS:
16     Mux16(a=a , b=b , sel=sel[0] , out=ab );
17     Mux16(a=c , b=d , sel=sel[0] , out=cd );
18     Mux16(a=ab , b=cd , sel=sel[1] , out=out );
19 }
```

NAND2Tetris / Hardware Simulator / Mux4Way16.hdl

HDL ☒ --LBSt Project 1 Mux4Way16 Chip Mux4Way16 Eval Reset Clock: 0

```
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/1/Mux4Way16.hdl
5 /**
6  * 4-way 16-bit multiplexor:
7  * out = a if sel = 00
8  *       b if sel = 01
9  *       c if sel = 10
10 *       d if sel = 11
11 */
12 CHIP Mux4Way16 {
13     IN a[16], b[16], c[16], d[16], sel[2];
14     OUT out[16];
15
16     PARTS:
17     Mux16(a=a , b=b , sel=sel[0] , out=ab );
18     Mux16(a=c , b=d , sel=sel[0] , out=cd );
19     Mux16(a=ab , b=cd , sel=sel[1] , out=out );
20 }
```

Input pins

a 0 0 0 1 0 0 1 0 0 0 1 1 0 1 0 0 dec

b 1 0 0 1 1 0 0 0 0 0 1 1 1 0 1 1 0 dec

c 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec

d 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 1 dec

sel 1 1 dec

Output pins

out 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec

Internal pins

ab 1 0 0 1 1 0 0 0 0 0 1 1 1 0 1 1 0 dec

Test Steps: 11 Outputs: 9 Slow Fast

Test Script Compare File Output File Diff Execution speed

```
39 eval,
40 output;
41
42 set sel 2,
43 eval,
44 output;
45
46 set sel 3,
47 eval,
48 output;
```

Simulation successful: The output file is identical to the compare file

13) MUX8WAY16

```
14) CHIP Mux8Way16 {
15)     IN a[16], b[16], c[16], d[16],
16)         e[16], f[16], g[16], h[16],
17)         sel[3];
18)     OUT out[16];
19)
20)     PARTS:
21)         Mux4Way16(a=a , b=b , c=c , d=d , sel=sel[0..1] ,
out=muxAD );
22)         Mux4Way16(a=e , b=f , c=g , d=h , sel=sel[0..1] ,
out=muxEH );
23)         Mux16(a=muxAD , b=muxEH , sel=sel[2] , out=out );
24) }
```

NAND2Tetris / Hardware Simulator / Mux8Way16.hdl

HDL ☒ --LBSt Project 1 Mux8Way16 Chip Mux8Way16 Eval Reset Clock: 0

```
5 /**
10 *   d if sel = 011
11 *   e if sel = 100
12 *   f if sel = 101
13 *   g if sel = 110
14 *   h if sel = 111
15 */
16 CHIP Mux8Way16 {
17     IN a[16], b[16], c[16], d[16],
18         e[16], f[16], g[16], h[16],
19         sel[3];
20     OUT out[16];
21
22     PARTS:
23         Mux4Way16(a=a , b=b , c=c , d=d , sel=sel[0..1] , out=muxAD );
24         Mux4Way16(a=e , b=f , c=g , d=h , sel=sel[0..1] , out=muxEH );
25         Mux16(a=muxAD , b=muxEH , sel=sel[2] , out=out );
26 }
```

Input pins

a	0 0 0 1 0 0 1 0 0 0 1 1 0 1 0 0	dec
b	0 0 1 0 0 0 1 1 0 1 0 0 0 1 0 1	dec
c	0 0 1 1 0 1 0 0 0 1 0 1 0 1 1 0	dec
d	0 1 0 0 0 1 0 1 0 1 1 0 0 1 1 1	dec
e	0 1 0 1 0 1 1 0 0 1 1 1 1 0 0 0	dec
f	0 1 1 0 0 1 1 1 1 0 0 0 1 0 0 1	dec
g	0 1 1 1 1 0 0 0 1 0 0 1 1 0 1 0	dec
h	1 0 0 0 1 0 0 1 1 0 1 0 1 0 1 1	dec

Test ☒ ☐ Steps: 19 Outputs: 17 Slow Fast

Test Script Compare File Output File Diff Table

```
79 eval,
80 output;
81
82 set sel 6,
83 eval,
84 output;
85
86 set sel 7,
87 eval,
88 output;
```

Simulation successful: The output file is identical to the compare file

14) DMUX4WAY

```
15) CHIP DMux4Way {
16)     IN in, sel[2];
17)     OUT a, b, c, d;
18)
19)     PARTS:
20)     DMux(in=in , sel=sel[1] , a=topHalf , b=bottomHalf );
21)     DMux(in=topHalf , sel=sel[0] , a=a , b=b );
22)     DMux(in=bottomHalf , sel=sel[0] , a=c , b=d );
23) }
```

NAND2Tetris / Hardware Simulator / DMux4Way.hdl

HDL --LBSt Project 1 DMux4Way Chip DMux4Way Eval Reset Clock: 0

```
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/1/DMux4Way.hdl
5 /**
6  * 4-way demultiplexor:
7  * [a, b, c, d] = [in, 0, 0, 0] if sel = 00
8  *               [0, in, 0, 0] if sel = 01
9  *               [0, 0, in, 0] if sel = 10
10 *               [0, 0, 0, in] if sel = 11
11 */
12 CHIP DMux4Way {
13     IN in, sel[2];
14     OUT a, b, c, d;
15
16     PARTS:
17     DMux(in=in , sel=sel[1] , a=topHalf , b=bottomHalf );
18     DMux(in=topHalf , sel=sel[0] , a=a , b=b );
19     DMux(in=bottomHalf , sel=sel[0] , a=c , b=d );
20 }
```

Input pins

in 1

sel 1 1 dec

Output pins

a 0

b 0

c 0

d 1

Internal pins

topHalf 0

bottomHalf 1

Test Steps: 11 Outputs: 9 Slow Fast

Test Script Compare File Output File Diff Table

```
33 eval,
34 output;
35
36 set sel %B10,
37 eval,
38 output;
39
40 set sel %B11,
41 eval,
42 output;
```

Simulation successful: The output file is identical to the compare file

15) DMUX8WAY

```
16 CHIP DMux8Way {
17     IN in, sel[3];
18     OUT a, b, c, d, e, f, g, h;
19
20     PARTS:
21     DMux(in=in , sel=sel[2] , a=topHalf , b=bottomHalf );
22     DMux4Way(in=topHalf , sel=sel[0..1] , a=a , b=b , c=c ,
23     d=d );
24     DMux4Way(in=bottomHalf , sel=sel[0..1] , a=e , b=f , c=g ,
25     d=h );
26 }
```

NAND2Tetris / Hardware Simulator / DMux8Way.hdl

HDL --LBSt Project 1 DMux8Way

Chip DMux8Way Eval Reset Clock: 0

Input pins

in 1

sel 1 1 1 dec

Output pins

a 0

b 0

c 0

d 0

e 0

f 0

g 0

h 0

Test Steps: 19 Outputs: 17 Slow Fast

Test Script Compare File Output File Diff Table

```
65 eval,
66 output;
67
68 set sel %B110,
69 eval,
70 output;
71
72 set sel %B111,
73 eval,
74 output;
```

Simulation successful: The output file is identical to the compare file

PROJECT 2

1) *HalfAdder*

```
2)  CHIP HalfAdder {
3)      IN a, b;      // 1-bit inputs
4)      OUT sum,      // Right bit of a + b
5)          carry;    // Left bit of a + b
6)
7)      PARTS:
8)      Xor(a=a, b=b, out=sum);
9)      And(a=a, b=b, out=carry);
10) }
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top toolbar includes icons for file operations, simulation control, and settings. The main window is divided into three sections:

- HDL Editor:** Shows the code for `HalfAdder.hdl`. The code defines a chip that takes two 1-bit inputs, `a` and `b`, and produces two outputs, `sum` and `carry`. It uses a `Xor` gate for the sum and an `And` gate for the carry.
- Chip HalfAdder:** Displays the current state of the chip's pins. Input pins `a` and `b` are both set to 1. Output pins `sum` and `carry` are set to 0 and 1, respectively.
- Test Script:** Shows a sequence of commands for testing the chip: `set a 1`, `set b 0`, `eval`, `output`, `set a 1`, `set b 1`, `eval`, and `output`.

At the bottom, a status bar indicates: "Simulation successful: The output file is identical to the compare file".

2) FullAdder

```
3)  CHIP FullAdder {
4)      IN a, b, c;  // 1-bit inputs
5)      OUT sum,      // Right bit of a + b + c
6)          carry;    // Left bit of a + b + c
7)
8)      PARTS:
9)      HalfAdder(a=a, b=b, sum=s1, carry=c1);
10)     HalfAdder(a=c, b=s1, sum=sum, carry=c2);
11)     Or(a=c1, b=c2, out=carry);
12) }
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top bar shows the file path: `NAND2Tetris / Hardware Simulator / FullAdder.hdl`. Below the toolbar, the `FullAdder` chip is selected, and its internal components are visible. The `CHIP FullAdder` block is shown with its input pins (`a`, `b`, `c`) and output pins (`sum`, `carry`). The internal pins (`s1`, `c1`, `c2`) are also visible. The simulation is running, and the output pins are set to 1, while the internal pins are set to 0.

The main window displays the HDL code for the `FullAdder` chip, which is a Verilog-like code. The code defines the chip's inputs, outputs, and internal components. The code is as follows:

```
1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/2/FullAdder.hdl
5 /**
6  * Computes the sum of three bits.
7  */
8 CHIP FullAdder {
9     IN a, b, c; // 1-bit inputs
10    OUT sum, // Right bit of a + b + c
11        carry; // Left bit of a + b + c
12
13    PARTS:
14    HalfAdder(a=a, b=b, sum=s1, carry=c1);
15    HalfAdder(a=c, b=s1, sum=sum, carry=c2);
16    Or(a=c1, b=c2, out=carry);
17 }
```

The bottom panel shows the test script, which is a sequence of commands used to test the chip. The test script is as follows:

```
37 output;
38
39 set b 1,
40 set c 0,
41 eval,
42 output;
43
44 set c 1,
45 eval,
46 output;
```

The simulation is successful, and the output file is identical to the compare file.

3) *Add16*

```
4)  CHIP Add16 {
5)      IN a[16], b[16];
6)      OUT out[16];
7)
8)      PARTS:
9)      HalfAdder(a=a[0], b=b[0], sum=out[0], carry=c1);
10)     FullAdder(a=a[1], b=b[1], c=c1, sum=out[1], carry=c
11)         2);
12)     FullAdder(a=a[2], b=b[2], c=c2, sum=out[2], carry=c
13)         3);
14)     FullAdder(a=a[3], b=b[3], c=c3, sum=out[3], carry=c
15)         4);
16)     FullAdder(a=a[4], b=b[4], c=c4, sum=out[4], carry=c
17)         5);
18)     FullAdder(a=a[5], b=b[5], c=c5, sum=out[5], carry=c
19)         6);
20)     FullAdder(a=a[6], b=b[6], c=c6, sum=out[6], carry=c
21)         7);
22)     FullAdder(a=a[7], b=b[7], c=c7, sum=out[7], carry=c
23)         8);
24)     FullAdder(a=a[8], b=b[8], c=c8, sum=out[8], carry=c
25)         9);
26)     FullAdder(a=a[9], b=b[9], c=c9, sum=out[9], carry=c
27)         10);
28)     FullAdder(a=a[10], b=b[10], c=c10, sum=out[10],
29)         carry=c11);
30)     FullAdder(a=a[11], b=b[11], c=c11, sum=out[11],
31)         carry=c12);
32)     FullAdder(a=a[12], b=b[12], c=c12, sum=out[12],
33)         carry=c13);
34)     FullAdder(a=a[13], b=b[13], c=c13, sum=out[13],
35)         carry=c14);
36)     FullAdder(a=a[14], b=b[14], c=c14, sum=out[14],
37)         carry=c15);
38)     FullAdder(a=a[15], b=b[15], c=c15, sum=out[15],
39)         carry=carry);
40) }
```

NAND2Tetris / Hardware Simulator / Add16.hdl

HDL ☐ --LBS ☐ Project 2 Add16

```
1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/2/Add16.hdl
5 /**
6  * 16-bit adder: Adds two 16-bit two's complement values.
7  * The most significant carry bit is ignored.
8  */
9 CHIP Add16 {
10     IN a[16], b[16];
11     OUT out[16];
12
13     PARTS:
14     ~HalfAdder(a=a[0], b=b[0], sum=out[0], carry=c1);
15     ~FullAdder(a=a[1], b=b[1], c=c1, sum=out[1], carry=c2);
16     ~FullAdder(a=a[2], b=b[2], c=c2, sum=out[2], carry=c3);
17     ~FullAdder(a=a[3], b=b[3], c=c3, sum=out[3], carry=c4);
18     ~FullAdder(a=a[4], b=b[4], c=c4, sum=out[4], carry=c5);
```

Chip Add16 Eval Reset Clock: 0

Input pins

a 0 0 0 1 0 0 1 0 0 0 1 1 0 1 0 0 dec

b 1 0 0 1 1 0 0 0 0 1 1 1 0 1 1 0 dec

Output pins

out 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 dec

Internal pins

c1 0

c2 0

c3 1

c4 0

c5 1

Test ☐ ☒ Steps: 9 Outputs: 7 Slow Fast

Test Script Compare File Output File Diff Table

```
29
30 set a %B0011110011000011,
31 set b %B0000111111110000,
32 eval,
33 output;
34
35 set a %B0001001000110100,
36 set b %B1001100001110110,
37 eval,
38 output;
```

Simulation successful: The output file is identical to the compare file

4) *Inc16*

```
5)    CHIP Inc16 {
6)        IN in[16];
7)        OUT out[16];
8)
9)        PARTS:
10)       Add16(a=in, b[0]=true, out=out);
11) }
```

The screenshot displays the NAND2Tetris Hardware Simulator interface. The top bar shows the project name "NAND2Tetris / Hardware Simulator / Inc16.hdl". The main window is divided into three sections:

- HDL Editor:** Contains the Verilog code for the Inc16 chip, which implements a 16-bit incrementer using the Add16 component.
- Chip Inc16:** A panel on the right showing the input and output pins. The input pin "in" is set to the binary value 111111111111011 (decimal 5), and the output pin "out" is set to 111111111111100 (decimal 6).
- Test:** A section at the bottom showing the test script and the simulation results. The test script includes commands to evaluate the chip's output for the given input. The results show that the output is identical to the compare file, indicating a successful simulation.

The test script and results are as follows:

```
15 eval,
16 output;
17
18 set in %B0000000000000101, // in = 5
19 eval,
20 output;
21
22 set in %B111111111111011, // in = -5
23 eval,
24 output;
```

Simulation successful: The output file is identical to the compare file

5) *ALU*

```
6)  CHIP ALU {
7)      IN
8)          x[16], y[16], // 16-bit inputs
9)          zx, // zero the x input?
10)         nx, // negate the x input?
11)         zy, // zero the y input?
12)         ny, // negate the y input?
13)         f, // compute (out = x + y) or (out = x & y)?
14)         no; // negate the out output?
15)      OUT
16)          out[16], // 16-bit output
17)          zr,      // if (out == 0) equals 1, else 0
18)          ng;      // if (out < 0) equals 1, else 0
19)
20)      PARTS:
21)          Mux16 (a=x, b=false, sel=zx, out=w1); // if (zx == 1)
set x = 0
22)          Mux16 (a=y, b=false, sel=zy, out=w2); // if (zy ==
1) set y = 0
23)
24)          // if (nx == 1) set x = !x
25)          Not16 (in=w1, out=notw1);
26)          Mux16 (a=w1, b=notw1, sel=nx, out=w3);
27)
28)          // if (ny == 1) set y = !y
29)          Not16 (in=w2, out=notw2);
30)          Mux16 (a=w2, b=notw2, sel=ny, out=w4);
31)
32)          // if (f == 1) set out = x + y else set out = x &
y
33)          And16 (a=w3, b=w4, out=xandy);
34)          Add16 (a=w3, b=w4, out=addxy);
35)          Mux16 (a=xandy, b=addxy, sel=f, out=w5);
36)
37)          // if (no == 1) set out = !out
38)          // if (out < 0) set ng = 1
39)          Not16 (in=w5, out=notw5);
40)          Mux16 (a=w5, b=notw5, sel=no, out=out,
out[0..7]=out0to7, out[8..15]=out8to15, out[15]=ng);
```

```

41)
42)         // if (out == 0) set zr = 1
43)     Or8Way (in=out0to7, out=or0to7);
44)     Or8Way (in=out8to15, out=or8to15);
45)     Or (a=or0to7, b=or8to15, out=or0to16);
46)     Not (in=or0to16, out=zr);
47) }

```

NAND2Tetris / Hardware Simulator / ALU.hdl

HDL --LBSt Project 2 ALU

Chip ALU Eval Reset Clock: 0

```

29 CHIP ALU {
44     Mux16 (a=x, b=false, sel=zx, out=w1); // if (zx == 1) set x = 0
56     And16 (a=w3, b=w4, out=xandy);
57     Add16 (a=w3, b=w4, out=addxy);
58     Mux16 (a=xandy, b=addxy, sel=f, out=w5);
59
60     // if (no == 1) set out = !out
61     // if (out < 0) set ng = 1
62     Not16 (in=w5, out=notw5);
63     Mux16 (a=w5, b=notw5, sel=no, out=out, out[0..7]=out0to7, out[8..15]=out8to15, out[16]=out16);
64
65     // if (out == 0) set zr = 1
66     Or8Way (in=out0to7, out=or0to7);
67     Or8Way (in=out8to15, out=or8to15);
68     Or (a=or0to7, b=or8to15, out=or0to16);
69     Not (in=or0to16, out=zr);
70 }

```

Input pins

x 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 de

y 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 de

zx 0

nx 1

zy 0

ny 1

f 0

no 1

Output pins

Test Steps: 41 Outputs: 37 Slow Fast

Test Script Compare File Output File Diff Table

```

365
366 // Compute x | y
367 set zx 0,
368 set nx 1,
369 set zy 0,
370 set ny 1,
371 set f 0,
372 set no 1,
373 eval,
374 output;

```

Simulation successful: The output file is identical to the compare file